VajraKavach

Attack Tolerance Report

Continuing to Operate Under Active Attack

Report Date:	April 7, 2026
Classification:	CONFIDENTIAL
Prepared By:	VajraKavach Security Team
Platform:	Secure Job Search & Professional Networking
Target:	`https://192.168.3.44`
Course:	CSE 345/545, IIIT-Delhi

Contents

1 Executive Summary **3**

2 Resource Bounding **3**
2.1 Memory Ceiling . . . 3
2.2 CPU Quota . . . 3
2.3 File Descriptor Limit . . . 3

3 Traffic Shaping Under Attack **4**
3.1 Dual-Layer Rate Limiting . . . 4
3.2 Connection Throttling . . . 4
3.3 HTTP 444 on Plain HTTP . . . 4

4 Progressive Degradation **5**
4.1 Progressive Lockout . . . 5
4.2 Graceful Error Responses . . . 5

5 Isolation & Independence **6**
5.1 Database Connection Pooling . . . 6
5.2 WebSocket Connection Isolation . . . 6
5.3 Async Architecture . . . 6

6 Periodic Cleanup **7**

7 Tolerance Summary **7**
7.1 Attack Scenario Matrix . . . 8

1 Executive Summary

Attack tolerance refers to the system's ability to **continue serving legitimate users while under active attack**. Unlike mitigation (which prevents attacks) or recovery (which restores service after attacks), tolerance ensures that a successful or ongoing attack against one component does not cause a complete service outage.

VajraKavach implements **10 distinct tolerance mechanisms** that bound the impact of attacks, limit attacker resource consumption, and maintain service availability for legitimate users throughout an attack scenario.

2 Resource Bounding

2.1 Memory Ceiling

Mechanism: `MemoryMax=2G` in systemd service configuration.
Location: `/etc/systemd/system/vajrakavach-backend.service`
Attack Scenario: An attacker sends requests designed to consume excessive memory (large payloads, many concurrent sessions, memory-intensive operations like blockchain mining).
Behavior Under Attack: If the backend process exceeds 2 GB of memory, systemd's cgroup controller kills it via `SIGKILL`. The service is then automatically restarted within 3 seconds (see Recovery report). During the restart, Nginx returns 502 Bad Gateway — but the **host system remains stable** and other services (PostgreSQL, Nginx, SSH) are unaffected.
Impact Containment: The attacker cannot exhaust host memory or trigger the OOM killer against other processes.

2.2 CPU Quota

Mechanism: `CPUQuota=80%` in systemd service configuration.
Location: `/etc/systemd/system/vajrakavach-backend.service`
Attack Scenario: CPU-intensive attacks such as triggering many blockchain proof-of-work computations, Argon2 password hashing, or cryptographic operations.
Behavior Under Attack: The backend process is throttled by the Linux CFS scheduler to use at most 80% of one CPU core. The remaining 20% is guaranteed available for system services, SSH access, and monitoring.
Impact Containment: System administrators can always SSH in and investigate, even during a CPU exhaustion attack.

2.3 File Descriptor Limit

Mechanism: `LimitNOFILE=65536` in systemd service configuration.
Location: `/etc/systemd/system/vajrakavach-backend.service`
Purpose: Allows the backend to handle a large number of concurrent connections (WebSocket, database, file operations) while preventing file descriptor exhaustion attacks from affecting the system. The limit is generous enough for normal operation but caps runaway resource consumption.

3 Traffic Shaping Under Attack

3.1 Dual-Layer Rate Limiting

Mechanism: Rate limiting at both Nginx (kernel-level) and FastAPI (application-level).
Attack Scenario: Brute-force login attempts, credential stuffing, API flooding.
Behavior Under Attack:

1. **Nginx layer (first line):** Rejects excess requests with HTTP 429 before they reach the Python process. This is extremely efficient — the request never enters userspace processing.
2. **FastAPI layer (second line):** Catches requests that bypass Nginx limits (e.g., from localhost). Provides endpoint-specific granularity.

Why both layers matter: Nginx handles the volumetric aspect (raw request count), while the application layer handles semantic rate limiting (specific endpoint patterns). An attacker cannot bypass one without hitting the other.
Legitimate user impact: Rate limits are per-IP. An attacker flooding from one IP address does **not** affect users on other IP addresses. Legitimate users experience zero degradation during a single-source flooding attack.

3.2 Connection Throttling

Mechanism: 15 concurrent connections per IP via Nginx `limit_conn`.
Attack Scenario: Slowloris attacks, connection pool exhaustion, SYN flooding.
Behavior Under Attack: After 15 connections from one IP, additional connection attempts are rejected immediately. This prevents a single attacker from monopolizing server resources. With Nginx's efficient connection handling, the server can maintain thousands of connections to legitimate users while rejecting excess connections from an attacker.
Impact Containment: Each attacker IP is bounded to 15 connections — far below the server's capacity.

3.3 HTTP 444 on Plain HTTP

Mechanism: Port 80 returns HTTP 444 (close connection immediately).
Location: `nginx/nginx.conf`
Attack Scenario: Automated scanners, bots, and reconnaissance tools probing port 80.
Behavior Under Attack: The TCP connection is closed without any response body. No server resources are spent generating error pages or redirect responses. This is the most resource-efficient way to handle unwanted traffic.
Impact Containment: Zero CPU/memory cost per rejected connection.

4 Progressive Degradation

4.1 Progressive Lockout

Mechanism: Exponentially increasing lockout for failed TOTP attempts.
Location: `backend/app/services/totp_service.py`
Attack Scenario: TOTP brute-force attack (an attacker who has obtained the password and is trying to guess the 6-digit TOTP code).
Behavior Under Attack:

Lockout Level	Duration	Cumulative Wait
1st lockout	60 minutes	1 hour
2nd lockout	300 minutes	6 hours
3rd lockout	1,500 minutes	31 hours
4th lockout	7,500 minutes	156 hours

Key design decision: Lockout is keyed by **IP address**, not email. This prevents an attacker from locking out a legitimate user by sending failed attempts against their email. The legitimate user can still log in from a different IP address.
Impact on legitimate users: Zero — lockout only affects the attacking IP.

4.2 Graceful Error Responses

Mechanism: Structured error responses with retry guidance.
Behavior Under Attack:

- Rate-limited clients receive HTTP 429 with message: `"Rate limit exceeded. Please try again later."`
- Auth-locked IPs receive HTTP 403 with lockout details
- Health endpoint (`/api/health`) always returns 200 regardless of attack state
- Frontend retry logic handles 429/503 with 3-second automatic backoff

Purpose: Legitimate clients can detect rate limiting and retry appropriately. The system never crashes or returns opaque 500 errors during attack handling.

5 Isolation & Independence

5.1 Database Connection Pooling

Mechanism: SQLAlchemy async connection pool with bounded size.
Location: `backend/app/database.py`
Configuration:

```
pool_size=10, max_overflow=5, pool_pre_ping=True
```

Attack Scenario: An attacker triggers many database-heavy operations to exhaust connections.
Behavior Under Attack: The pool limits concurrent database connections to 15 (10 base + 5 overflow). Excess requests queue and eventually timeout rather than crashing. `pool_pre_ping=True` validates connections before use, handling stale connections from database restarts.
Impact Containment: PostgreSQL is never overwhelmed by connection storms.

5.2 WebSocket Connection Isolation

Mechanism: Independent authentication and lifecycle for each WebSocket connection.
Location: `backend/app/routes/websocket.py`
Features:

- Each connection is independently authenticated via JWT
- Dead connections are automatically cleaned up and discarded
- Connection registry uses asyncio lock for thread safety
- A compromised or malicious connection cannot affect other users' connections
- Connections are organized per-user with set-based storage (efficient add/remove)

Attack Scenario: An attacker opens many WebSocket connections to exhaust resources.
Behavior Under Attack: Each connection requires valid JWT authentication. Invalid tokens are rejected at connection time (close code 4001/4003). The per-IP connection limit of 15 (Nginx) bounds the maximum connections from one attacker.

5.3 Async Architecture

Mechanism: FastAPI with Python asyncio event loop.
Purpose: All I/O operations (database queries, WebSocket communication, file operations) are non-blocking. A slow request (e.g., blockchain mining, large file encryption) does not block the event loop or prevent other requests from being processed.
Attack Scenario: An attacker sends requests that trigger slow operations (complex queries, large file processing).
Behavior Under Attack: Other users' requests continue to be processed concurrently. The async architecture naturally isolates the impact of slow requests.

6 Periodic Cleanup

Mechanism: Automatic cleanup of rate limit and lockout data structures.
Location: `backend/app/middleware/security.py` (rate limiter), `backend/app/services/auth_service.py` (token blocklist)
Attack Scenario: A distributed attack from many IP addresses fills the in-memory rate limit dictionary.
Behavior Under Attack:

- Rate limiter: Entries older than 60 seconds are pruned every 60 seconds
- Token blocklist: Expired tokens are cleaned up at startup and periodically
- TOTP lockout: Stale entries with expired lockout windows are naturally reset

Purpose: Prevents memory growth from distributed attacks where thousands of unique IPs each send a few requests. Without cleanup, the dictionaries would grow indefinitely.

7 Tolerance Summary

#	Tolerance Mechanism	Attack Type Bounded
1	Memory ceiling (2 GB)	Memory exhaustion, DoS
2	CPU quota (80%)	CPU exhaustion, cryptomining
3	File descriptor limit (65536)	Connection exhaustion
4	Dual-layer rate limiting	Flooding, brute-force
5	Connection throttling (15/IP)	Slowloris, connection pooling
6	HTTP 444 on plaintext	Scanner probes, HTTP floods
7	Progressive IP lockout	TOTP brute-force
8	Database connection pool (10+5)	Connection storm, query flooding
9	WebSocket isolation	WS connection exhaustion
10	Async architecture	Slow request attacks
Total Tolerance Mechanisms	**10**	

Key design principle: Every tolerance mechanism is designed to **bound the impact per-attacker** without affecting legitimate users on different IP addresses. The system degrades gracefully for the attacker while maintaining full service for everyone else.

7.1 Attack Scenario Matrix

Attack	System Response	Legitimate User Impact
HTTP Flood (single IP)	Rate limited at Nginx (429), then at app layer	None (per-IP limiting)
Distributed HTTP Flood	Rate limited per-IP; cleanup prevents memory growth	Minimal latency increase
Slowloris	15 connection limit per IP	None
TOTP Brute-Force	Progressive lockout (60 min → 300 min → ...)	None (IP-keyed, not email-keyed)
Memory Exhaustion	Process killed at 2 GB, auto-restart in 3s	3-second service interruption
CPU Exhaustion	Throttled to 80% CPU	20% CPU available for system/SSH
Database Connection Storm	Pool limited to 15; excess requests queue	Slight latency increase
WebSocket Flood	JWT auth required; 15 connections per IP max	None